```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define N 4

void b(int slot, int etat[N])  // opérateur b détruisant une particule sur le slot désiré
{
    etat[slot]=0;
}

void bdag (int slot, int etat[N])   // opérateur bdag créant une particule sur le slot désiré
{
    etat[slot]=1;
}

int puiss2 (int nombre)  // calcul de 2 à la puissance nombre
{
    int i, puissance=1;
    for (i=0;i<nombre;i++)
    {
        puissance = puissance * 2;
    }
    return puissance;
}

int puiss10 (int nombre)  // calcul de 10 à la puissance nombre
{
    int i, puissance=1;
    for (i=0;i<nombre;i++)
    {
        puissance = puissance * 10;
    }
    return puissance;
}


 void transformation_binaire (int nombre, int *etat) //fonction prenant en argument un nombre et un vecteur et qui renvoie un vecteur contenant le nombre bianair correspondant
{
    int reste, i;
    for (i=0; i<N; i++)
    {
        *(etat+i)=0;
    }
    int tour=1;
    while (nombre > 0)  // on effectue des divisions euclidiennes successives par 2, on remplit le vecteur par les restes de ces divisions euclidiennes
    {
        reste=nombre%2;
        etat[N-tour] = reste ;
        nombre=nombre/2;
        tour=tour+1;
    }
}

int transformation_decimal (int *binaire) // fonction prenant pour argument un vecteur de 0 et de 1 et renvoyant le nombre correspondant en base 10
{
    int puissance=N-1, nombre=0, i;
    for (i=0;i<N;i++)
    {
        nombre+=binaire[i]*puiss2(puissance);
        puissance-=1;
    }
    return nombre;
}
```

```c
void construction_base (int n)  // cette fonction permet de construire une base de l'ensemble étudié
{

                                // il s'agit d'une base canonique
                                //Cette fonction ne sera pas utilisée dans la suite du programme pour des raisons d'optimisation de mémoire
                                // elle permet une meilleure compréhension de la suite
    int i,j;
    for (i=0; i < n ; i++)
    {
        int *etat = malloc(N*sizeof(int));  //free dans la fonction
        transformation_binaire(i, etat);
        for (j=0;j<N;j++)
        {
            printf("%d",etat[j]);
        }
        printf("\n");
        free(etat);
    }
}


double ** matrice_hamiltonien(int n)   //cette fonction prend pour argument un entier n et renvoit la matrice carrée de taille n x n de l'hamiltonien
{
    int i, j, k, m, q, o;
    double ** etat_h = malloc(n*sizeof(double *));  //free dans la fonction valeur_propre, liste de pointeurs
    for (i=0;i<n;i++)
    {
        etat_h[i]=malloc(n*sizeof(double));         //free dans la fonction valeur_propre
        for (j=0;j<n;j++)
        {
            etat_h[i][j]=0;
        }
    }
                                    //initialisation à 0 de la matrice etat_h qui va contenir des 0 et des -1

    for (k=0;k<n;k++)               // on parcourt la matrice verticalement
    {
        int *saver_bis = malloc(N*sizeof(int)); //free dans la fonction
        transformation_binaire(k, saver_bis);
                                      // saver contient successivement les n vecteurs de taille n de la base

        for (m=0;m<N;m++)           //on parcourt chaque vecteur afin de tester les changements de localisation des électrons possibles
        {
                int indice_bis=0;

                if ((saver_bis[(m+N+1)%N]==1) && (saver_bis[m]==0))
                {                               //on teste si l'orbitale de départ contient un électron et si l'orbitale d'arrivée n'en contient pas
                    b((m+N+1)%N, saver_bis);    //si c'est le cas on peut déplacer l'électron de la première orbitale dans la seconde en utilisant les fonctions b et bdag
                    bdag(m, saver_bis);

                    indice_bis = transformation_decimal(saver_bis); // on récupère le nombre décimal associé au nouvel état atteint

                    etat_h[indice_bis][k]=-1;                       // on remplit la matrice avec un coefficient -1 à la position adéquate
```

```c

                    bdag((m+N+1)%N, saver_bis);

                    b(m, saver_bis);                                // on revient au vecteur initial

                }
                if ((saver_bis[(m+N+1)%N]==0) && (saver_bis[m]==1))
                                                //on teste si l'orbitale de départ contient "rien" et si l'orbitale d'arrivée contient un électrou
                {                               //si c'est le cas on peut déplacer l'électron de la seconde orbitale dans la première en utilisant les fonctions b et bdag
                    bdag((m+N+1)%N, saver_bis);
                    b(m, saver_bis);

                    indice_bis = transformation_decimal(saver_bis); // on récupère le nombre décimal associé au nouvel état atteint

                    etat_h[indice_bis][k]=-1;                       // on remplit la matrice avec un coefficient -1 à la position adéquate

                    b((m+N+1)%N, saver_bis);                        // on revient au vecteur initial

                    bdag(m, saver_bis);


                }
        }
        free (saver_bis);                   //libère la mémoire de saver_bis
    }
    return(etat_h);                         //on libère la mémoire allouée à la matrice

}



void jacobi(double **matrix, double eps, int n) // prend en argument un pointeur double, le critère d'arrêt et le nombre d'états
//méthode de diagonalisation de la matrice
{
    int p, q, r, x,y,m,o;
    double m_rp, m_rq, m_qq, m_pp, m_pq ;
    double maximum; // initialisation de la variable contenant la coordonnée non diagonale maximale

    for (m=0;m<n;m++) //on imprime la matrice del'hamiltonien pour s'assurer de sa cohérence
    {
        for (o=0;o<n;o++)
        {
            printf("%+.0f ", matrix[m][o]);
        }
        printf("\n");
    }
    printf("\n");


    do // tant que arret est plus grand que epsilon, on rentre dans le for
    {
        for (p=0; p<n; p++)
        {
            for (q=0; q<n; q++)
            {
                double phi = (0.5*atan((2*matrix[p][q])/(matrix[q][q]-matrix[p][p])));

                if ((q!=p) && fabs(matrix[p][q]) >= eps)
                {


```

```c
                    for (r=0; r<n; r++)
                    {
                        if (r!=p && r != q)  // application des relations de Jacobi pour les éléments non diagonaux
                        {
                            m_rp = matrix[r][p];
                            m_rq = matrix[r][q];

                            matrix[r][p]=m_rp*cos(phi)-m_rq*sin(phi);
                            matrix[r][q]=m_rq*cos(phi)+m_rp*sin(phi);

                            matrix[p][r] = matrix[r][p] ;
                            matrix[q][r] = matrix[r][q] ;
                        }
                    }

                    m_pq = matrix[p][q]; // la matrice est symétrique
                    m_pp = matrix[p][p];
                    m_qq = matrix[q][q];

                    // application des relations de Jacobi pour les éléments diagonaux
                    matrix[p][p] = m_pp * cos(phi)*cos(phi) + m_qq*sin(phi)*sin(phi) - 2* m_pq * cos(phi)*sin(phi);
                    matrix[q][q] = m_pp * sin(phi)*sin(phi) + m_qq*cos(phi)*cos(phi) + 2* m_pq * cos(phi)*sin(phi);

                    matrix[p][q]= 0.0 ;
                    matrix[q][p]= 0.0 ;
                }

            }
        }

        maximum = 0.0 ;

        for (x=0;x<n;x++) // actualisation du maximum des éléments non diagonaux de la matrice
        {
            for (y=0;y<n;y++)
            {
                if (x != y && maximum <= fabs(matrix[x][y]))
                {
                    maximum = fabs(matrix[x][y]) ;
                }
            }
        }
    } while (maximum >= eps) ;

    for (m=0;m<n;m++) // affichage de la matrice diagonalisée
    {
        for (o=0;o<n;o++)
        {
            printf("%+.3f ", matrix[m][o]);
        }
        printf("\n");
    }
}



double *valeurs_propres(int n)
{
    int i, m, l;
    double **h = matrice_hamiltonien(n) ;
    jacobi(h, 1.0e-12, n);

    double *vp_energies = malloc(n*sizeof(double));
```

```c

    for (i=0;i<n;i++)
    {
        vp_energies[i] = h[i][i] ;
    }
    for (int m=0;m<n;m++)
    {
            printf("%+.3f ", vp_energies[m]);
    }
        printf("\n");

    for (l=0;l<n;l++)
    {
        free(h[l]);    //libère la mémoire de chaque tableau contenu dans la matrice h
    }
    free (h);          //libère la mémoire des pointeurs des tableaux précédents
    return vp_energies ;
}

double calcul_energie (double T,int n, double vp_energies[n])
// calcul de l'énergie interne du système à partir des valeurs propres de l'hamiltonien dans la base des états
{
    int i ;
    double moy_E = 0.0, numerateur = 0.0, denominateur = 0.0 ;
    for (i=0;i<n;i++)
    {
        const double E = vp_energies[i] ; // const permet de s'assurer qu'on ne modifie pas la valeur de E dans la boucle
        const double et = exp(-E/T) ;
        numerateur += E*et;
        denominateur += et;
    }

    moy_E = numerateur/denominateur;
    return(moy_E);
}


double ** matrice_hamiltonien_2(int n) // ceci est une ébauche de solution pour la suite du problème
// problèmes identifiés : méthode non efficace
{
    int i, j, k, m,p,q,r, compteur;
    double ** base = malloc(n*sizeof(double *));
    for (i=0;i<n;i++)
    {
        base[i]=malloc(n*sizeof(double));
        for (j=0;j<n;j++)
        {
            base[i][j]=0;
        }
    }

    for (p=0;p<n;p++)
    {
        compteur=0;
        int *vecteur = malloc(N*sizeof(int));
        transformation_binaire (p, vecteur);
        for (q=0;q<N;q++)
        {
            if (vecteur[q]==1)
            {
                compteur+=1;
            }
        }
        if (compteur ==2)
```

```
        {
            for (r=0;r<N;r++)
            {
                base[p][r]=vecteur[r];
            }
        }
    }

    double ** etat_h = malloc(n*sizeof(double *));
    for (i=0;i<n;i++)
    {
        etat_h[i]=malloc(n*sizeof(double));
        for (j=0;j<n;j++)
        {
            etat_h[i][j]=0;
        }
    }

    for (k=0;k<n;k++) // on parcourt la matrice de la base des états
    {
        int *saver_bis = base[k];
        for (m=0;m<N;m++)
        {
                if ((saver_bis[(m+N+1)%N]==1) && (saver_bis[m]==0))
                {
                    b((m+N+1)%N, saver_bis);
                    bdag(m, saver_bis);
                    int valeur = transformation_decimal(saver_bis);
                    int s = 0;
                    while (valeur != transformation_decimal(base[s]))
                    {
                        s+=1;
                    }
                    etat_h[s][k]=-1;

                    bdag((m+N+1)%N, saver_bis);
                    b(m, saver_bis);

                }
                if ((saver_bis[(m+N+1)%N]==0) && (saver_bis[m]==1))
                {
                    bdag((m+N+1)%N, saver_bis);
                    b(m, saver_bis);

                    int valeur = transformation_decimal(saver_bis);
                    int s = 0;
                    while (valeur != transformation_decimal(base[s]))
                    {
                        s+=1;
                    }
                    etat_h[s][k]=-1;

                    b((m+N+1)%N, saver_bis);
                    bdag(m, saver_bis);


                }
        }
    }
        return(etat_h);

}

double *valeurs_propres_2(int n)
{
    int i, m, k;
```

```
    double **h = matrice_hamiltonien_2(n) ;
    jacobi(h, 1.0e-12,n);

    double *vp_energies = malloc(n*sizeof(double));

    for (i=0;i<n;i++)
    {
        vp_energies[i] = h[i][i] ;
    }
    for (k=0;k<puiss2(N);k++)
    {
        free(h[k]);
    }
    free(h);

    printf("\n");
    for (int m=0;m<n;m++)
    {
            printf("%+.3f ", vp_energies[m]);
    }
    printf("\n");

    return vp_energies ;
}




int main()
{
    int T_indice ;
    int n = puiss2(N);
    double T = 0.0 ;
    double energie = 0.0 ;
    double tlog ;
    const int nt = 100 ;

    double *v_p = malloc(n*sizeof(double));
    v_p = valeurs_propres(n);
    for (T_indice=0 ; T_indice<nt ; T_indice++)
    {
        tlog = log(1.0e-1) + T_indice*(log(1.0e2) - log(1.0e-1))/(nt - 1) ;
        T = exp(tlog) ;
        energie = calcul_energie(T,n, v_p);
        printf("%f\n", energie);
    }
    free(v_p);


    int taille = 6;
    double *v_p_2 = malloc(taille*sizeof(double));
    v_p_2 = valeurs_propres_2(taille);
    for (T_indice=0 ; T_indice<nt ; T_indice++)
    {
        tlog = log(1.0e-1) + T_indice*(log(1.0e2) - log(1.0e-1))/(nt - 1) ;
        T = exp(tlog) ;
        energie = calcul_energie(T,taille, v_p);
        printf("%f\n", energie);
    }
    free(v_p_2);
    return 0 ;
}
```